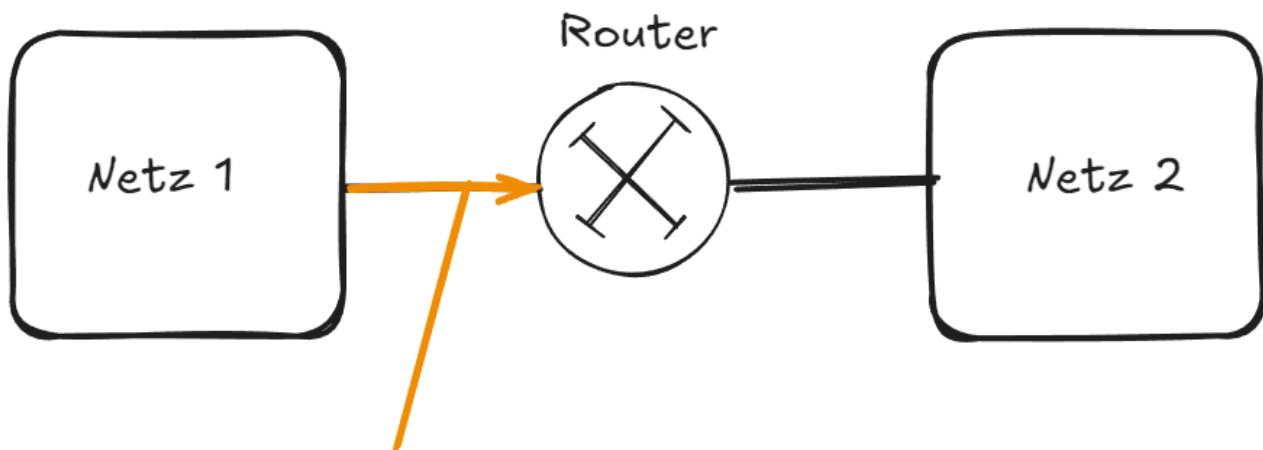


Theorie (ergänzt nach der UE)

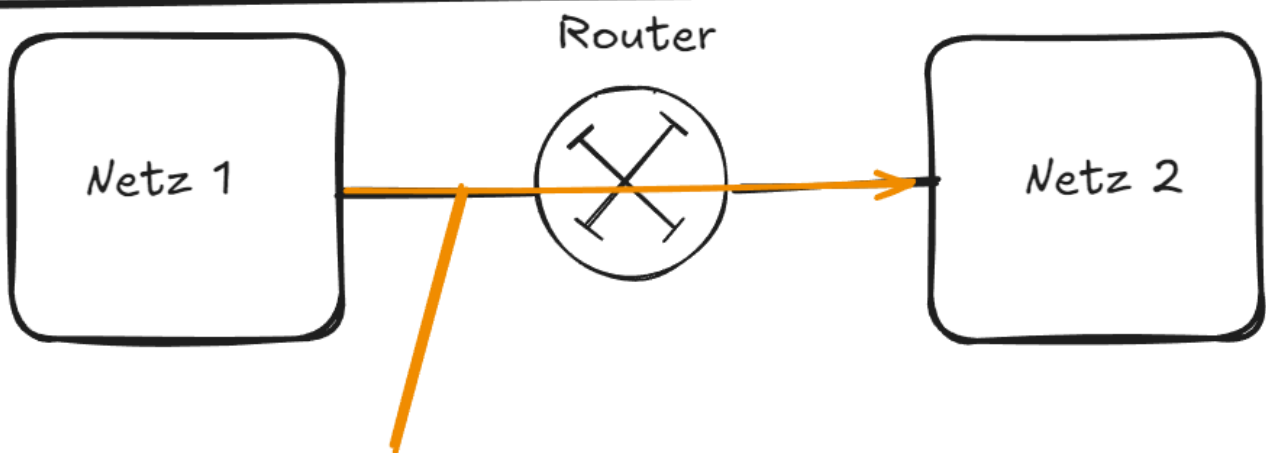
Die Idee von Filtering

(Um diese Übung gut zu verstehen, sollten zuerst die theoretischen Teile der vorherigen Übung (Routing wiederholt werden)

Ein Router erlaubt es, Pakete von einem Netzwerk in ein anderes zu übertragen. Die Pakete, welche von Netzwerk 1 nach Netzwerk 2 wollen, kommen beim Router an, und dieser schaut in seinem IPTable, ob er weiß, in welches Netzwerk die Pakete wollen. Kennt er dieses (oder kennt er es nicht und hat einen Default Gateway) so sendet er diese weiter.

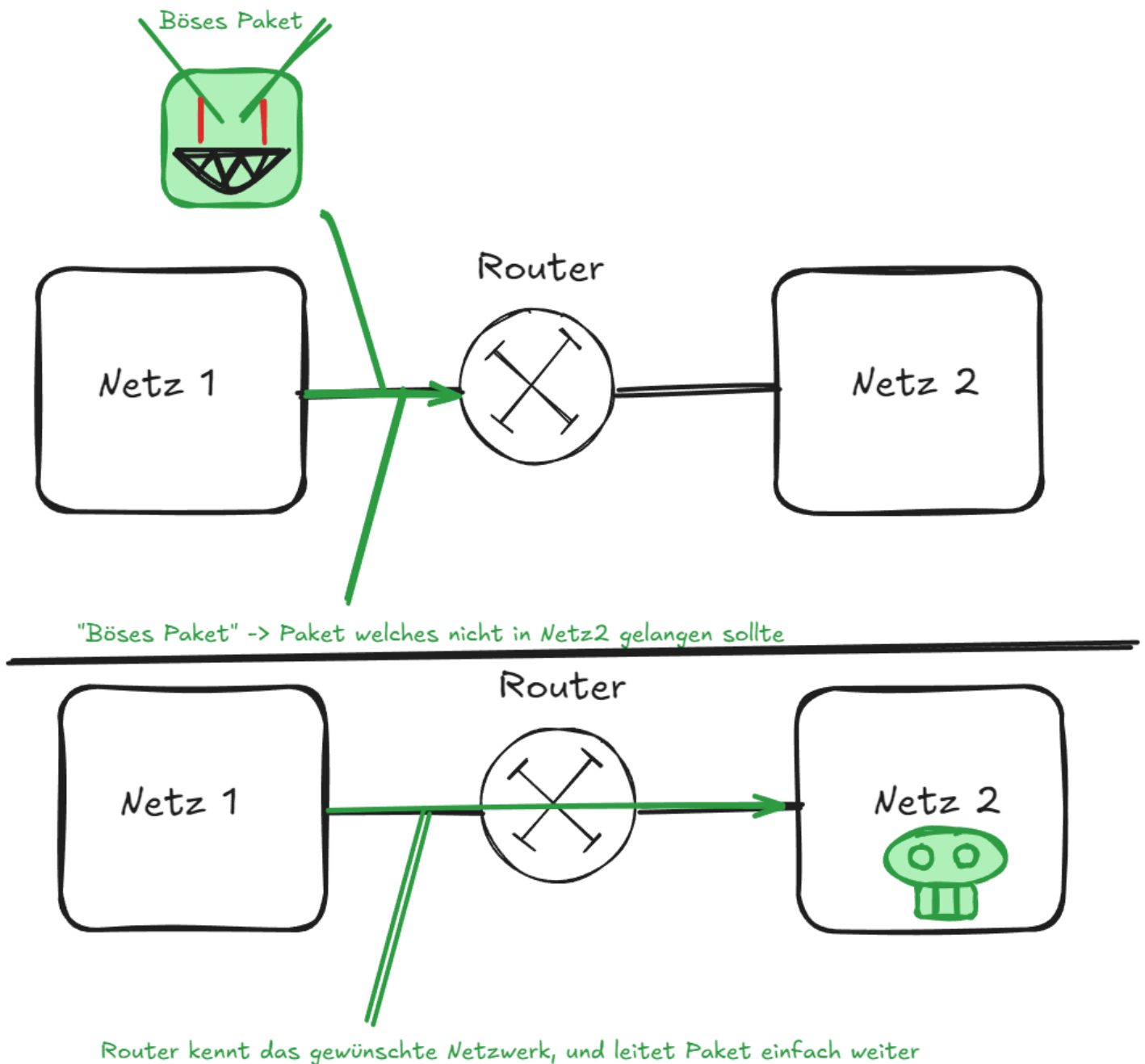


Paket mit Herkunftsadresse in Netz1 und Zieladresse in Netz 2



Router kennt Netz2, und leitet das Paket hindurch

Dieser Vorgang ist so prinzipiell gewünscht. Aber wenn ein Netzwerk sensible Daten hat, oder jemand ein Paket in böser Absicht schickt, passiert eben genau das gleiche. Das böse Paket wird hindurchgeleitet und kommt in das Zielnetzwerk.



Dies will man verhindern. Da man jedoch unterscheiden muss, welche Pakete gut und welche Böse sind, muss man sie vor dem Hindurchlassen überprüfen. Dieses Überprüfen ist das **Filtering**

Filtering erlaubt es, Regeln für verschiedene Pakete aufzustellen. Diese Regeln werden vor dem Hinein, Hinaus oder Durchlassen durch das System geprüft. Hierbei gibt es prinzipiell 2 Arten des Filterings:

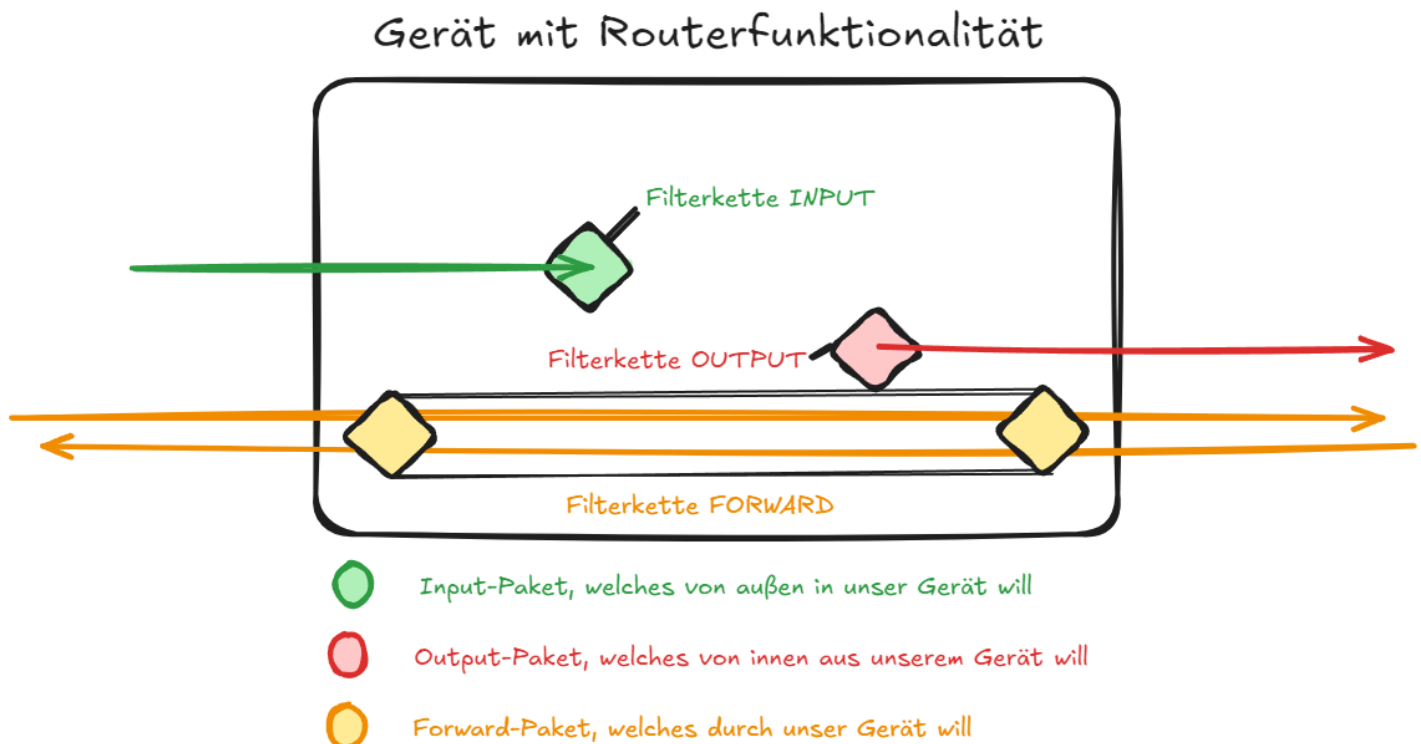
1. **Blacklisting:** Alles ist erlaubt, außer eine Regel verbietet es. Diese Vorgehensweise ist offen und praktisch, weil nicht für jedes Paket eine eigene Zusatzregel erstellt werden muss, allerdings ist das Risiko auch höher. Diese Philosophie findet eher bei Endgeräten Anwendung, da diese bereits durch Whitelist Router oder ähnliches geschützt sein sollten.
2. **Whitelisting:** Alles ist verboten, außer eine Regel erlaubt es.** Diese Vorgehensweise ist einschränkender, aber sicherer. Sie wird eher bei Geräten verwendet, welche als Router oder Zwischengeräte verwendet werden, um unbefugten Zugriff auf Netzwerke vorzubeugen.

Der Aufbau der Filterregeln

FORWARD, INPUT, und OUTPUT

Grundsätzlich gibt es drei verschiedene Arten von Paketen.

- Pakete, welche von außen kommen und das eigene System als Ziel haben, sind **INPUT-Pakete**
- Pakete, welche vom eigenen System kommen und nach außen wollen, sind **OUTPUT-Pakete**
- Pakete, welche von außen kommen und zu einem anderen System wollen, sind **FORWARD-Pakete**

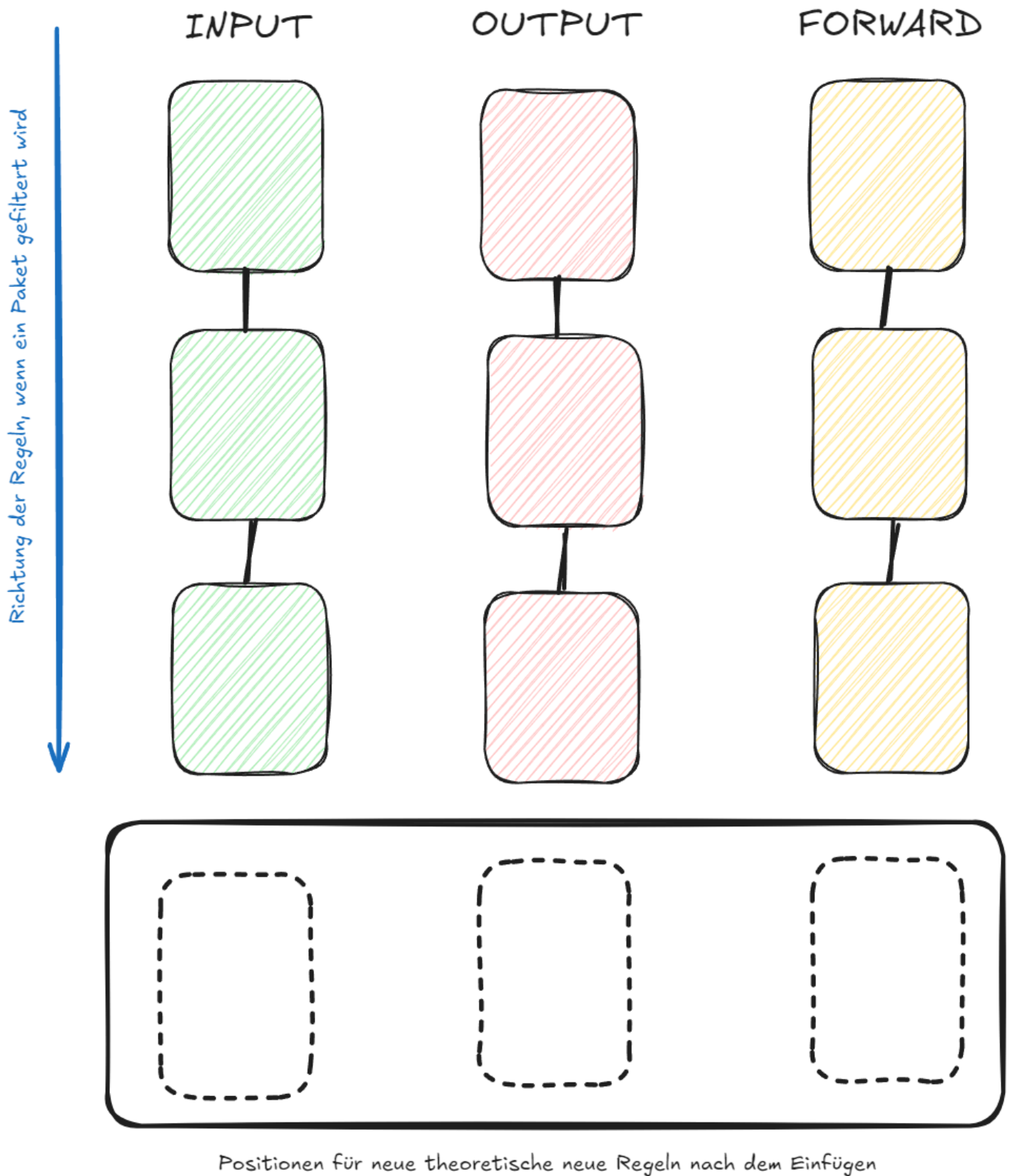


Jede dieser Möglichkeiten besitzt eine eigene Liste (genannt Kette) an Regeln. Diese Regeln können auf Linux mit dem iptables Befehl diesen bestimmten Ketten zugewiesen werden.

```
``iptables -A FORWARD -p icmp -j ACCEPT
```

Dieser Befehl setzt beispielsweise die Regel (setzen =append, deswegen -A), dass in der Kette FORWARD alle Pakete des Protokolls(-p) icmp durchgelassen werden sollen (-j ACCEPT)

Wichtig zu beachten: Neue Regeln werden immer "**unten**" in der Kette eingefügt. Die Kette wird aber beim Abfragen von **oben nach unten** durchgegangen. Sobald eine Regel gefunden wird, welche auf das Paket zutrifft, wird diese verwendet und die anderen Regeln werden **ignoriert**. Das bedeutet, dass die spezifischen Regeln immer als erstes eingefügt werden müssen, um nicht von generelleren Regeln überschrieben zu werden.



Diese Filter betrachten die statischen Eigenschaften der Pakete, also wo sie herkommen und hinwollen. Nun kann man aber auch andere Faktoren betrachten, zB. ob die Pakete einer Reihe von Paketen angehören, oder einer aufgebauten Verbindung. Dies hat den enormen Vorteil, dass man anstatt für alle Pakete, welche in einem Datenaustausch vorkommen könnten, eigene Regeln setzen zu müssen, stattdessen nur Regeln für die ersten Pakete, welche die Verbindung herstellen gebraucht werden, und der Rest einfach durchgeschleust werden kann.

Dieses Vorgehen nennt man **Stateful Inspection**. Regeln werden hier nicht für alle Pakete einer Kette, sondern für Pakete, welche einen bestimmten Status haben, gesetzt. Das erspart massiv

Arbeit, da nun ganze zusammenhängende Kategorien von Paketen bearbeitet werden können, und nicht jede Paketart einzeln bearbeitet werden muss.

Dies geschieht unter Linux über `ip_conntrack`.

Ablauf - Mitschrift

Im ersten Schritt werden die beiden linken Rechner per Kabel verbunden.

Dann wird

```
sudo -s und nmcli net off
```

verwendet um das automatische routing zu deaktivieren.

Dann wird eine manuelle verbindung eingestellt.

Am rechten Rechner wird die Verbindung über

```
ifconfig eth0 10.20.5.x/24 (wobei x = pc nummer)
```

eingestellt.

Es wird eine Verbindung zum mittleren Rechner (Router) hergestellt. Der linke Rechner (Kabelgebunden) ist über `usb0` (Kabel) verbunden, der rechte über das Labornetz (`eth0`)

Dann wird das Routing auf dem mittleren Rechner mittels

```
echo 1 > / proc / sys / net / ipv4 / ip_forward
```

aktiviert.

Im Anschluss werden die default interfaces / default gateways gesetzt. Dies geschieht über den Befehl:

```
route add default gw <ip-address>
```

Das wichtigste ist, dass der linke und rechte Rechner sich am Ende pinggen können.

Dann haben wir die Korrektheit der Netzmasken geprüft, indem wir das Keyword `netmask` in der Ausgabe des

```
ifconfig
```

 Befehls mit `255.255.255.0` abgeglichen haben.

Dann werden die Ketten geprüft

```
iptables -L
```

Hier sieht man bei `POLICY`, welche policy gesetzt sind, welche standardmäßig `ACCEPT` sein sollte. Dies passiert über

```
iptables --policy <chain> <target> //valide targets stehen im Hilfsfile
```

```
iptables --policy INPUT -j DROP
```

Das Ergebnis ist, dass sich die äußersten Rechner noch pinggen können. Allerdings kann der mittlere Rechner nicht erreicht werden. Pingt der mittlere Rechner, geht es zwar nach außen, aber die Antwort wird verworfen.

Danach wurden alle weiteren (`OUTPUT`, `FORWARD`) ebenfalls auf `DROP` gesetzt.

```
iptables --policy INPUT -j DROP
iptables --policy OUTPUT -j DROP
```

```
iptables --policy FORWARD -j DROP
```

Im WireShark sollten die Anfragen noch angezeigt werden. (Wichtig, richtiges Interface abhören!)

Jetzt soll ausschließlich das pingen erlaubt werden.

Dies passiert über:

```
iptables -A FORWARD -p icmp -j accept
```

Da der Befehl nur für FORWARD gilt, kann nur der rechte / linke den jeweils anderen erreichen.

Die neue Regel lässt sich in `iptables -L` anzeigen

`iptables -F` (f = flush) löscht alle Regeln, also alle Ketten, löschen. Einzig die Default-Policy bleibt erhalten. Diese muss immer händisch geändert werden.

Setzt man die selbe Regel für INPUT und OUTPUT, sollten wieder pings in alle Richtungen zwischen allen Geräten funktionieren

Es können auch einzelne Regeln herausgelöscht werden, mit

```
iptables -D FORWARD -p icmp -j accept
```

(Achtung, der Rest muss komplett ident sein!)

Aufgabe 3 - Stateful Inspection

Status Established -> Verbindung aufgebaut (TCP), aber auch bei ICMP verwendbar. Diese Stati werden verwendet, um selektiv Pakete reinzulassen oder abzublocken, je nachdem was für einen Status die Pakete haben (zB. man erlaubt nur Pakete, die eine Antwort auf einen vorher gesendeten Request sind.)

Dadurch kann recht einfach gefiltert werden, es muss nur die Originalaussendung bekannt sein.

Dies ist die Stärke von Stateful Inspection.

die States lassen sich per `-m <state>` setzen.

wichtig sind hier vor allem ESTABLISHED, RELATED (eben jene um Antworten zu filtern)

Besonders wichtig sind diese States für Router (also den mittleren PC)

Damit kann man einstellen, dass eine Verbindung nur einseitig funktioniert. Dies funktioniert, indem man mit den Argumenten `-s` (sender) und `-d` (empfänger) als eigene Regeln so setzt, dass es von rechts nach links geht, aber nicht links nach rechts (oder vice versa, je nach eingestellter Richtung)

Befehle:

```
iptables -A FORWARD -p icmp -s <ipsender (a)> -d <ipempf (b)> -j ACCEPT  
iptables -A FORWARD -p icmp -s <ipsender (b)> -d <ipempf (a)> -j DROP
```

Dadurch das alle Antworten durchgehen, aber nur eine Richtung eine Anfrage schicken kann, kann nur in einer Richtung kommuniziert werden. RELATED betrifft die Antworten, ESTABLISHED muss eine aufgebaute Verbindungen (das geht auch auf Antworten von zB. TCP-Request (er weiß selber nicht genau was RELATED heißt eig.))

Aufgabe 4/5 SSH-Verbindung zu einem externen Server

Auf dem linken PC (internes Netz) wird ein SSH Server gestartet. Das Ziel ist, diesen vom Rechner rechts aus zu erreichen. Der Server kann mit

```
/etc/init.d/ssh start
```

gestartet werden.

Da die Verbindung zwischen den PCs nur einseitig funktioniert, kann die Verbindung darüber nicht getestet werden.

Auf dem mittleren Rechner muss nun eine Regel angelegt werden, welche nur Anfragen von rechts nach links durchlässt, und zwar nur auf dem Standard SSH-Port (22)

```
iptables -A FORWARD -p TCP --dport 22 -s <ipsender (a)> -d <ipempf (b)> -j ACCEPT
```

Das hier ist jetzt eher das Ergebnis für Aufgabe 5 (Lehrer messed up)

Theorie - Mitschrift

IP-Tables

Unter Linux wird das IPTables Programm verwendet, um den PC in verschiedenen Arten zu konfigurieren. Am wichtigsten ist hierbei der **Filtertable** (ist der Default-Table)

IPTables ist ein mächtiges Programm, wobei wir heute nicht mehr als den default sehen werden. Für den Filtertable gibt es die Ketten

Ketten

Ketten deswegen, weil sie eine **Kette an Regeln** sind, welche hintereinander geprüft werden, bis die erste gefunden wird die greift, der Rest der Kette wird dann ignoriert. (basically großes IF)

Dementsprechend müssen spezifische Regeln vor allgemeineren Regeln gesetzt werden. Dann kann ich systematisch nach Wahrscheinlichkeit filtern.

Spezifisch ---> Allgemein

Das allgemeinste ist die Default-Regel bzw. die **Policy**, welche garnicht in der Kette liegt und immer existiert. Das hat den Sinn, dass auch bei fehlender Kette zumindest eine Regel existiert.

Die erste Überlegung ist also immer, was die Policy sein soll.

```
ACCEPT -> Lässt alles rein
```

```
DROP -> Schmeißt alles weg
```

Welche besser ist, hängt von der Verwendung des Gerätes ab

Endgerät -> meistens ACCEPT

Router -> eher DROP

INPUT

- Betrifft alle Pakete, die direkt zum Gerät gehen

OUTPUT

- Betrifft alle Pakete, die vom Gerät ausgehen

FORWARD

- Betrifft alle Pakete, welches über das Gerät gehen (Ursprung und Ende != Gerät)

Die Filter machen eher nur auf dem Router Sinn. Bei einem Ping von links nach rechts geht der Ping über das mittlere Gerät (FORWARD). Filtert man FORWARD, ginge der Ping nicht mehr. Allerdings könnten links und rechts immernoch Mitte pingen.

Sperrt der Router die INPUT, kann er keine Daten mehr empfangen?

Die Regeln lassen sich so konfigurieren, dass nur in eine Richtung gepingt werden kann.

Regeln lassen sich auf einzelne Protokolle anlegen, und auch nur auf eigene ports anwenden

Shelly Mitschrift

- gibt verschiedene Filter
- diese Tables haben mehrere Chains (INPUT, FORWARD, OUTPUT) -> verwendet man um das Filtering zu machen
- ist defaulttable
- ist der wichtigste
- gibt 3 chains
- chains -> kette von verschiedenen Regeln
- INPUT -> alle Pakete filter, welche alle Pakete an die Firewall/System schickt
- OUTPUT -> alle Pakete weg von der Firewall/System
- FORWARD -> Pakete welche weitergeleitet werden -> wichtigste
- iptables -L -> kann sich anschauen welche chains man hat (selbst ohne konfiguration)
- werden von oben nach unten durchgelaufen -> wo bedingung erfüllt wird ausgeführt
- default policy -> policy accept -> wenn keine Regel zieht bzw keine Regel gibt -> automatisch default policy
- iptables -P -> kann man default policy ändern
- iptables -P INPUT DROP -> policy wird auf drop geändert
- security mäßig nicht ideal, dass default alles accept ist, allerdings beim endgerät feiner, denn wenn alles auf DROP wäre müsste man alles extra einstellen und aufmachen
- bei Firewall prinzipiell alles auf DROP
- iptables -f -> steht für flasch (???) -> löscht regeln heraus, aber die policy bleibt
- default policy ist außerhalb der kette/liste (?)
- die erste die zieht wird ausgeführt
- regel besteht aus iptables -A (Regel hinzufügen) Bedingungen -j (jump, Sprungmarke markieren, wo man hinspringen will) verdict (was mach ich mit dem Paket, einfach: DROP(nay) oder ACCEPT (yay))
- wenn mehrere angehängt -> wird unten angehängt
- wenn alles auf DROP -> alles blockiert
- man soll gezielt die dinge aufmachen, welche man erreichen möchte, zb Pings durchlassen
- siehe Aufgabe 2 erster Codesnippet
- ping verwendet icmp Pakete
- mittlerer Rechner kann nicht gepingt werden, weil INPUT & OUTPUT noch auf drop sind und die einzelne Regel nicht gesetzt ist

Verbindungsstatus evaluieren -> viel helfen, wenn antwortpakete zulasse -> filtert nach Antwortpakete, wenn Anfrage durchgeht soll auch Antwort durchgehen, nur rest zu beschränken, brauche nicht jede Kommunikation nicht einzeln filtern, wenn Anfrage durchgeht auch Antwort durchgehen

- man Schaut sich deshalb den Verbindungsstatus durch, hat verschiedene Zustände (im Moodle gibt es eine Allgemeine Liste zu den IP-Tables)
- ESTABLISHED wichtigstes, verbindung aufgebaut, alle pakete de zu established dazu gehören • - m state -> kann man sich status anschauen
- wenn man --state ESTABLISHED, RELATED verwendet -> funktioniert um status abzurufen
- wenn mehrere regeln verwedent werden und diese immer neue appended werden, und eine regel von oben schon zieht, dann wird immer schon die vorherige Regel verwendet werden und nie eine neue (z.B. gleiche regel aber mit kleiner änderung wird hinzugefügt -> kann niemals erreicht werden)
- sehr spezifische Regeln weiter oben und allgemeineeren Regeln unten -> weil sonst spezifische Regeln ignoriert werdne
- regel löschen: iptabels -D FORWARD -p -icmp -j ACCEPT -> kann man so spezeifische regeln löschen (z.B sehr allgemeine regel oben und dann kann man sie unten weider einfügen)
- man muss Source & Destination eingeben um speziellerere Regeln einzugeben (iptables - FORWARDD -p icmp -j ACCEPT ist zu allgemein) -> iptables -A FORWARD -p icmp -s (source) 169.254.0.2 -d (destination) 10.20.5.5 (bspw.) -j ACCEPT